

Synthèse Projet ESP32-S3

Affichage graphique bas-niveau · ST7920 SBC-LCD128×64

Version 2 · SPI Hardware — SPI_MODE3 confirmé

1 · Matériel & Environnement

1.1 Microcontrôleur — ESP32-S3-DevKitC-1-N8R8

Puce	ESP32-S3 (Xtensa LX7 dual-core, 240 MHz)
Flash	8 Mo (Octal SPI)
PSRAM	8 Mo (Octal SPI – OPI)
RAM interne	512 Ko SRAM
Wi-Fi	802.11 b/g/n 2.4 GHz
Bluetooth	BLE 5.0
MAC address	Obtenue via esp_read_mac() au démarrage
USB	USB-OTG natif + UART (CH340)
IDE	Arduino IDE 2.x – esp32 board package

1.2 Écran — SBC-LCD128×64 (contrôleur ST7920)

Résolution	128 × 64 pixels (monochrome)
Contrôleur	Sitronix ST7920
Interface	SPI série 3 fils (mode Serial Peripheral)
Alimentation	VCC = 5 V – rétroéclairage BLA = 5 V / BLK = GND
Mémoire	DDRAM (texte) + GDRAM (graphique 128×64 bits)

2 · Câblage SPI (bit-bang)

2.1 Correspondance des broches

Broche LCD	Rôle	GPIO ESP32-S3	Remarque
------------	------	---------------	----------

RS (CS)	Chip Select	GPIO 10	Actif HIGH pour parler au ST7920
R/W (MOSI)	Data In (master→slave)	GPIO 11	Données série envoyées bit à bit
E (SCK)	Horloge	GPIO 12	Front montant = échantillonnage
PSB	Mode série/parallèle	GND	GND = mode série SPI obligatoire
VCC	Alimentation	5 V	Tension logique 5 V
GND	Masse	GND	
BLA	Rétroéclairage +	5 V	Résistance série recommandée
BLK	Rétroéclairage -	GND	

■ Le niveau logique des GPIO ESP32-S3 est 3,3 V. Le ST7920 accepte des entrées 3,3 V même alimenté en 5 V dans la plupart des modules.

2.2 Protocole SPI bit-bang

Le matériel SPI de l'ESP32-S3 n'est pas utilisé : chaque bit est produit manuellement via **digitalWrite**. Le ST7920 exige l'envoi de **nibbles** (demi-octets) : d'abord les 4 MSB, puis les 4 LSB.

```
sendByte(0xF8);          // marqueur commande
sendByte(cmd & 0xF0);    // nibble haut (bits 7-4)
sendByte(cmd << 4);      // nibble bas (bits 3-0 décalés en 7-4)
```

3 · Architecture Logicielle

3.1 Organisation des fichiers

st7920.h	Déclarations publiques, constantes GPIO, struct Caractere
st7920.cpp	Implémentation : init, SPI, GDRAM, pixels, formes, texte
police_standard.h	Table ASCII 0x20-0x7E, bitmaps 8x8 pixels
main.cpp	setup() / loop() – point d'entrée Arduino

3.2 Buffer GDRAM logiciel

Un tableau `uint8_t gdrum[64][16]` (= 1 024 octets) est maintenu en RAM. Toute modification de pixel met à jour ce buffer puis envoie la paire d'octets correspondante au ST7920 (le contrôleur exige des mots de 16 bits).

```
uint8_t gdrum[64][16]; // 64 lignes × 16 octets = 128 pixels/ligne
int col = x / 8;        // numéro de l'octet dans la ligne
int bit = 7 - (x % 8); // position du bit (MSB = pixel le plus à gauche)
```

3.3 Particularité d'adressage ST7920

L'écran est divisé en deux demi-écrans de 32 lignes. Le câblage fait correspondre 0x80 aux lignes 0–31 (haut) et 0x88 aux lignes 32–63 (bas).

```
// Partie haute (y < 32)
sendCommand(0x80 | y);          // adresse verticale
sendCommand(0x80 | (col/2));    // adresse horizontale (mots 16 bits)
// Partie basse (y >= 32)
sendCommand(0x80 | (y - 32));
sendCommand(0x88 | (col/2));
// Envoi du mot de 16 bits
sendData(gdrum[row][col & ~1]); // octet pair (MSB)
sendData(gdrum[row][col | 1]);  // octet impair (LSB)
```

4 · Référence des Fonctions

st7920_init()

Initialise les GPIO, envoie la séquence d'initialisation ST7920 (mode basique → étendu → graphique), vide le buffer gdram.

```
sendCommand(0x30); // mode basique
sendCommand(0x34); // mode étendu
sendCommand(0x36); // mode graphique (GDRAM)
```

effacerEcran()

Parcourt les 64 lignes et envoie 0x00 sur les 16 octets de chaque ligne.

```
for (int y = 0; y < 64; y++)
    for (int x = 0; x < 16; x++) sendData(0x00);
```

setPixel(x, y, allumer)

Fonction centrale : allume ou éteint un pixel via OR / AND-NOT dans gdram, puis rafraîchit le mot de 16 bits sur l'écran.

```
gdram[row][col] |= (1 << bit); // allumer
gdram[row][col] &= ~(1 << bit); // éteindre
```

tracerLigne(x1,y1,x2,y2)

Algorithme de Bresenham : ligne droite par additions entières uniquement.

```
int erreur = dx - dy;
// avance en x ou y selon le signe de l'erreur accumulée
```

tracerRectangle(x1,y1,x2,y2)

Quatre appels à tracerLigne() pour les côtés haut, bas, gauche, droite.

```
tracerLigne(x1,y1,x2,y1); tracerLigne(x1,y2,x2,y2);
tracerLigne(x1,y1,x1,y2); tracerLigne(x2,y1,x2,y2);
```

tracerCercle(cx,cy,r)

Bresenham cercle : 8 points symétriques tracés simultanément par octant.

```
allumerPixel(cx+x,cy+y); allumerPixel(cx-x,cy+y);
// ... 6 autres octants ...
```

afficherCaractere(c, x, y)

Recherche le code ASCII dans police_standard[], dessine le bitmap 8×8.

```
for (int ligne = 0; ligne < 8; ligne++) {
    uint8_t octet = police_standard[i].bitmap[ligne];
    for (int bit = 0; bit < 8; bit++)
        if (octet & (0x80 >> bit)) allumerPixel(x+bit, y+ligne);
}
```

afficherTexte(texte, x, y)

Affiche une chaîne C caractère par caractère à partir de (x, y). Retour à la ligne automatique si débordement à droite (revient à x de départ). S'arrête silencieusement si le bas de l'écran est atteint (y + 8 > 64). Recommandation : limiter les chaînes à 16 caractères max pour éviter tout mélange.

```
int cx = x, cy = y;
for (int i = 0; texte[i] != '\0'; i++) {
    if (cx + 8 > 128) { cx = x; cy += 8; } // retour à la ligne
    if (cy + 8 > 64) return;               // plus de place en bas
    afficherCaractere(texte[i], cx, cy);
    cx += 8;
}
```

5 · Police de Caractères

5.1 Structure Caractere

```
struct Caractere {
    uint8_t code;        // code ASCII
    uint8_t bitmap[8];   // 8 lignes × 8 pixels
};
```

5.2 Couverture ASCII

Plage couverte	0x20 (espace) → 0x7E (~) – 94 caractères imprimables
Manquant	0x60 (accent grave) – pas de lettre accentuée (Latin-1)
Chaque glyphe	8×8 pixels – bit 7 = pixel gauche, bit 0 = pixel droit
Espacement	Largeur fixe 8 px, hauteur fixe 8 px

5.3 Exemple — lettre 'A' (0x41)

```
{0x41, {0x18, // 00011000 . . . X X . . .
        0x24, // 00100100 . . X . . X . .
        0x42, // 01000010 . X . . . . X .
        0x42, // 01000010 . X . . . . X .
        0x7E, // 01111110 . X X X X X X . <- barre
        0x42, // 01000010 . X . . . . X .
        0x42, // 01000010 . X . . . . X .
        0x00}} // 00000000 . . . . . . . .
```

6 · Programme de Test (main.cpp)

Affiche trois chaînes de texte à des positions fixes. Chaque chaîne est limitée à **16 caractères maximum** (16 × 8 px = 128 px) pour éviter tout débordement et mélange de lignes.

```
#include "st7920.h"
void setup() {
    st7920_init(); effacerEcran();
    // max 16 caractères par chaîne (16 × 8px = 128px = largeur écran)
    afficherTexte("Hello ESP32!", 0, 0);
    afficherTexte("Ligne 2 texte", 0, 8);
    afficherTexte("0123456789ABCDEF", 0, 16);
}
void loop() {}
```

7 · Points d'Amélioration

- **afficherTexte(str, x, y)** ■ — fait. Chaînes limitées à 16 caractères pour éviter les mélanges.
- **Caractères accentués** ■ — fait. police_accents.h avec codes > 0x7F, séquences \x80, \x81... dans les chaînes.
- **SPI hardware** ■ — fait. SPI_MODE3 confirmé par test matériel. Voir section 8.
- **Dirty flags** — ne rafraîchir que les mots de 16 bits modifiés. Prochaine étape avant le scrolling.
- **Scrolling horizontal** — faire défiler un texte de droite à gauche en boucle.
- **Formes remplies** — tracerRectanglePlein(), tracerCerclePlein() par scan-line.
- **Largeur variable des glyphes** — i, l, ' plus étroits. Refactoring complet de la struct Caractere.
- **LED RGB NeoPixel (GPIO 38)** — signaler les états : rouge = erreur, vert = OK, bleu = transmission.

8 · Passage au SPI Hardware — Réalisé et Validé

8.1 Pourquoi le bit-bang était limitant

Dans la version 1, chaque bit était envoyé manuellement via `digitalWrite()` avec des délais de 1 μ s entre chaque front d'horloge. Un rafraîchissement complet de l'écran (1024 octets) prenait environ **50 ms**, soit ~10-20 fps maximum — insuffisant pour un scrolling fluide.

8.2 Ce qui a changé

Le passage au SPI hardware a été **chirurgical** : seules trois fonctions bas niveau ont été modifiées. Toute la logique au-dessus est restée identique.

Fonction	Avant (bit-bang)	Après (SPI hardware)	Impact
<code>sendByte()</code>	8 <code>digitalWrite()</code> + délais	supprimée	disparaît
<code>sendCommand()</code>	3× <code>sendByte()</code>	3× <code>SPI.transfer()</code>	modifiée
<code>sendData()</code>	3× <code>sendByte()</code>	3× <code>SPI.transfer()</code>	modifiée
<code>st7920_init()</code>	<code>pinMode()</code> × 3	<code>SPI.begin()</code> + <code>beginTransaction()</code>	modifiée
<code>setPixel()</code>	inchangée	inchangée	■ rien
<code>tracerLigne()</code>	inchangée	inchangée	■ rien
<code>afficherTexte()</code>	inchangée	inchangée	■ rien

8.3 Découverte importante — SPI_MODE3

Le datasheet du ST7920 (page 44) indique un timing compatible avec **SPI_MODE0** (CPOL=0, CPHA=0). Cependant, le test matériel a révélé que le module SBC-LCD128×64 utilisé requiert **SPI_MODE3** (CPOL=1, CPHA=1 — horloge idle HIGH). Ce comportement est probablement lié au module lui-même plutôt qu'au chip ST7920.

```
// Dans st7920_init() :
SPI.beginTransaction(SPISettings(
  1000000, // 1 MHz — max ST7920 = 2.5 MHz (datasheet p.43)
  MSBFIRST, // bit de poids fort en premier
  SPI_MODE3 // confirmé par test matériel (MODE0 du datasheet ne fonctionne pas)
));
```

8.4 Gain mesuré

Débit bit-bang	~500 Kbits/s effectif (avec délais 72 μ s entre chaque envoi)
Débit SPI hw	1 Mbits/s configuré – extensible jusqu'à 2.5 MHz
<code>sendByte()</code> supprimée	8 × <code>digitalWrite()</code> + <code>delayMicroseconds()</code> éliminés par octet
Résultat	Affichage confirmé fonctionnel – base prête pour le scrolling

Annexes · Code Source Complet

st7920.h

Déclarations publiques & struct Caractere

```
#ifndef ST7920_H
#define ST7920_H
#include <Arduino.h>

#define SCK 12
#define MOSI 11
#define CS 10

struct Caractere
{
    uint8_t code;
    uint8_t bitmap[8];
};

void st7920_init();
void effacerEcran();
void setPixel(int x, int y, bool allumer);
void allumerPixel(int x, int y);
void eteindrePixel(int x, int y);
void tracerLigne(int x1, int y1, int x2, int y2);
void tracerRectangle(int x1, int y1, int x2, int y2);
void tracerCercle(int cx, int cy, int r);
void afficherCaractere(uint8_t c, int x, int y);
void afficherTexte(const char* texte, int x, int y);

#endif
```

st7920.cpp

Implémentation complète du driver

[illegible]

```

void st7920_init()
{
    memset(gdram, 0, sizeof(gdram)); // Tous les bits de gdram sont mis à zéro. memset vient de string.h

    // Initialisation du bus SPI hardware
    // SPI.begin(SCK, MISO, MOSI, SS)
    // MISO = -1 car le ST7920 en mode série est write-only (pas de lecture possible)
    SPI.begin(SCK, -1, MOSI, CS);
    SPI.beginTransaction(SPISettings(
        1000000, // 1 MHz – prudent, max ST7920 = 2.5 MHz à 4.5V (datasheet p.43)
        MSBFIRST, // bit de poids fort en premier, comme en bit-bang (i=7 downto 0)
        SPI_MODE3 // CPOL=1 CPHA=1 : horloge idle HIGH, échantillonnage front descendant
        // NB: le datasheet indique MODE0 mais le test matériel a confirmé MODE3
    ));

    // SPI.begin() configure SCK, MOSI et CS automatiquement en OUTPUT
    // On force CS LOW pour s'assurer que le slave ne lit rien pendant l'init
    pinMode(CS, OUTPUT);
    digitalWrite(CS, LOW);

    delay(100); // Stabilisation de l'alimentation
    sendCommand(0x30); delay(2); // 0x30 = Mode basique
    sendCommand(0x0C); delay(2); // 0x0C = Allume l'affichage
    sendCommand(0x01); delay(10); // 0x01 = Efface la mémoire texte (DDRAM)
    sendCommand(0x30); delay(2); // 0x30 = Retour mode basique. Recommandation du datasheet
    sendCommand(0x34); delay(2); // 0x34 = Mode étendu. Passage obligatoire pour accéder au GDRAM
    sendCommand(0x36); delay(2); // 0x36 = Mode graphique
    // N.B. Pour afficher uniquement du texte, le mode basique (DDRAM) est suffisant.
    // Mais pour travailler au niveau des pixels il faut travailler en mode graphique (GDRAM)
}

// Le ST7920 partage l'écran en deux: une partie gauche et une partie droite.
// Mais le câblage fait que la partie gauche est affichée en haut et la droite en bas.
void effacerEcran()
{
    for (int y = 0; y < 64; y++)
    {
        if (y < 32)
        {
            sendCommand(0x80 | y);
            sendCommand(0x80);
        }
        else
        {
            sendCommand(0x80 | (y - 32));
            sendCommand(0x88);
        }
        for (int x = 0; x < 16; x++)
            sendData(0b00000000); // 0 = pixel éteint
    }
}

void setPixel(int x, int y, bool allumer)
{
    int row = y;
    int col = x / 8;
    int bit = 7 - (x % 8);
    if (allumer)
        gdram[row][col] |= (1 << bit);
    else
        gdram[row][col] &= ~(1 << bit);
    if (row < 32)
    {
        sendCommand(0x80 | row);
        sendCommand(0x80 | (col / 2));
    }
    else
    {
        sendCommand(0x80 | (row - 32));
        sendCommand(0x88 | (col / 2));
    }
    sendData(gdram[row][col & ~1]);
    sendData(gdram[row][col | 1]);
}

void allumerPixel(int x, int y) { setPixel(x, y, true); }

```

```

void eteindrePixel(int x, int y) { setPixel(x, y, false); }

void tracerLigne(int x1, int y1, int x2, int y2)
{
    int dx = abs(x2 - x1), dy = abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    int erreur = dx - dy;
    while (true)
    {
        allumerPixel(x1, y1);
        if (x1 == x2 && y1 == y2) break;
        int e2 = 2 * erreur;
        if (e2 > -dy) { erreur -= dy; x1 += sx; }
        if (e2 < dx) { erreur += dx; y1 += sy; }
    }
}

void tracerRectangle(int x1, int y1, int x2, int y2)
{
    tracerLigne(x1, y1, x2, y1); // haut
    tracerLigne(x1, y2, x2, y2); // bas
    tracerLigne(x1, y1, x1, y2); // gauche
    tracerLigne(x2, y1, x2, y2); // droite
}

void tracerCercle(int cx, int cy, int r)
{
    int x = 0, y = r;
    int d = 3 - 2 * r;
    while (x <= y)
    {
        allumerPixel(cx + x, cy + y); allumerPixel(cx - x, cy + y);
        allumerPixel(cx + x, cy - y); allumerPixel(cx - x, cy - y);
        allumerPixel(cx + y, cy + x); allumerPixel(cx - y, cy + x);
        allumerPixel(cx + y, cy - x); allumerPixel(cx - y, cy - x);
        if (d < 0) d += 4 * x + 6;
        else { d += 4 * (x - y) + 10; y--; }
        x++;
    }
}

void afficherCaractere(uint8_t c, int x, int y)
{
    // On cherche d'abord dans la police standard
    int nbStandard = sizeof(police_standard) / sizeof(police_standard[0]);
    for (int i = 0; i < nbStandard; i++)
    {
        if (police_standard[i].code == c)
        {
            for (int ligne = 0; ligne < 8; ligne++)
            {
                uint8_t octet = police_standard[i].bitmap[ligne];
                for (int bit = 0; bit < 8; bit++)
                {
                    if (octet & (0x80 >> bit))
                        allumerPixel(x + bit, y + ligne);
                }
            }
            return;
        }
    }
    // Si pas trouvé, on cherche dans la police des accents
    int nbAccents = sizeof(police_accents) / sizeof(police_accents[0]);
    for (int i = 0; i < nbAccents; i++)
    {
        if (police_accents[i].code == c)
        {
            for (int ligne = 0; ligne < 8; ligne++)
            {
                uint8_t octet = police_accents[i].bitmap[ligne];
                for (int bit = 0; bit < 8; bit++)
                {
                    if (octet & (0x80 >> bit))
                        allumerPixel(x + bit, y + ligne);
                }
            }
            return;
        }
    }
}

```



```

    // Caractère non trouvé : on n'affiche rien
}

void afficherTexte(const char* texte, int x, int y)
{
    int cx = x;
    int cy = y;
    for (int i = 0; texte[i] != '■'; i++)
    {
        if (cx + 8 > 128) { cx = x; cy += 8; }
        if (cy + 8 > 64) return;
        afficherCaractere((uint8_t)texte[i], cx, cy);
        cx += 8;
    }
}

```

police_standard.h

Table de police ASCII 8x8 (94 glyphs)

```

#ifndef POLICE_STANDARD_H
#define POLICE_STANDARD_H
#include "st7920.h"

const Caractere police_standard[] = {
    {0x20, {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00}}, // espace
    {0x21, {0x08,0x08,0x08,0x08,0x08,0x00,0x08,0x00}}, // !
    {0x22, {0x24,0x24,0x24,0x00,0x00,0x00,0x00,0x00}}, // "
    {0x23, {0x24,0x24,0x7E,0x24,0x7E,0x24,0x24,0x00}}, // #
    {0x24, {0x08,0x3E,0x48,0x3C,0x0A,0x7C,0x08,0x00}}, // $
    {0x25, {0x62,0x64,0x08,0x08,0x10,0x26,0x46,0x00}}, // %
    {0x26, {0x30,0x48,0x48,0x30,0x4A,0x44,0x3A,0x00}}, // &
    {0x27, {0x08,0x08,0x08,0x00,0x00,0x00,0x00,0x00}}, // '
    {0x28, {0x04,0x08,0x10,0x10,0x10,0x08,0x04,0x00}}, // (
    {0x29, {0x20,0x10,0x08,0x08,0x08,0x10,0x20,0x00}}, // )
    {0x2A, {0x00,0x08,0x2A,0x1C,0x2A,0x08,0x00,0x00}}, // *
    {0x2B, {0x00,0x08,0x08,0x3E,0x08,0x08,0x00,0x00}}, // +
    {0x2C, {0x00,0x00,0x00,0x00,0x00,0x08,0x08,0x10}}, // ,
    {0x2D, {0x00,0x00,0x00,0x3E,0x00,0x00,0x00,0x00}}, // -
    {0x2E, {0x00,0x00,0x00,0x00,0x00,0x00,0x08,0x00}}, // .
    {0x2F, {0x02,0x04,0x04,0x08,0x10,0x20,0x40,0x00}}, // /
    {0x30, {0x3C,0x42,0x46,0x4A,0x52,0x62,0x3C,0x00}}, // 0
    {0x31, {0x08,0x18,0x08,0x08,0x08,0x08,0x1C,0x00}}, // 1
    {0x32, {0x3C,0x42,0x02,0x0C,0x30,0x40,0x7E,0x00}}, // 2
    {0x33, {0x3C,0x42,0x02,0x1C,0x02,0x42,0x3C,0x00}}, // 3
    {0x34, {0x04,0x0C,0x14,0x24,0x7E,0x04,0x04,0x00}}, // 4
    {0x35, {0x7E,0x40,0x40,0x7C,0x02,0x42,0x3C,0x00}}, // 5
    {0x36, {0x1C,0x20,0x40,0x7C,0x42,0x42,0x3C,0x00}}, // 6
    {0x37, {0x7E,0x02,0x04,0x08,0x10,0x20,0x20,0x00}}, // 7
    {0x38, {0x3C,0x42,0x42,0x3C,0x42,0x42,0x3C,0x00}}, // 8
    {0x39, {0x3C,0x42,0x42,0x3E,0x02,0x04,0x38,0x00}}, // 9
    {0x3A, {0x00,0x00,0x08,0x00,0x00,0x08,0x00,0x00}}, // :
    {0x3B, {0x00,0x00,0x08,0x00,0x00,0x08,0x08,0x10}}, // ;
    {0x3C, {0x04,0x08,0x10,0x20,0x10,0x08,0x04,0x00}}, // <
    {0x3D, {0x00,0x00,0x7E,0x00,0x7E,0x00,0x00,0x00}}, // =
    {0x3E, {0x20,0x10,0x08,0x04,0x08,0x10,0x20,0x00}}, // >
    {0x3F, {0x3C,0x42,0x02,0x0C,0x08,0x08,0x00,0x00}}, // ?
    {0x40, {0x3C,0x42,0x4E,0x52,0x4E,0x40,0x3C,0x00}}, // @
    {0x41, {0x18,0x24,0x42,0x42,0x7E,0x42,0x42,0x00}}, // A
    {0x42, {0x7C,0x42,0x42,0x7C,0x42,0x42,0x7C,0x00}}, // B
    {0x43, {0x1E,0x20,0x40,0x40,0x40,0x20,0x1E,0x00}}, // C
    {0x44, {0x78,0x44,0x42,0x42,0x42,0x44,0x78,0x00}}, // D
    {0x45, {0x7E,0x40,0x40,0x7C,0x40,0x40,0x7E,0x00}}, // E
    {0x46, {0x7E,0x40,0x40,0x78,0x40,0x40,0x00,0x00}}, // F
    {0x47, {0x1E,0x20,0x40,0x4E,0x42,0x22,0x1C,0x00}}, // G
    {0x48, {0x42,0x42,0x42,0x7E,0x42,0x42,0x42,0x00}}, // H
    {0x49, {0x3E,0x08,0x08,0x08,0x08,0x08,0x3E,0x00}}, // I
    {0x4A, {0x3E,0x02,0x02,0x02,0x02,0x44,0x38,0x00}}, // J
    {0x4B, {0x42,0x4C,0x70,0x60,0x70,0x4C,0x42,0x00}}, // K
    {0x4C, {0x40,0x40,0x40,0x40,0x40,0x40,0x7E,0x00}}, // L
    {0x4D, {0x42,0x66,0x5A,0x5A,0x42,0x42,0x42,0x00}}, // M
    {0x4E, {0x42,0x62,0x52,0x4A,0x46,0x42,0x42,0x00}}, // N
    {0x4F, {0x3C,0x42,0x42,0x42,0x42,0x42,0x3C,0x00}}, // O
    {0x50, {0x7C,0x42,0x42,0x7C,0x40,0x40,0x40,0x00}}, // P
    {0x51, {0x3C,0x42,0x42,0x42,0x4A,0x44,0x3A,0x00}}, // Q
    {0x52, {0x7C,0x42,0x42,0x7C,0x48,0x44,0x42,0x00}}, // R

```

```

{0x53, {0x1E,0x20,0x40,0x3C,0x02,0x02,0x7C,0x00}}, // S
{0x54, {0x7F,0x08,0x08,0x08,0x08,0x08,0x08,0x00}}, // T
{0x55, {0x42,0x42,0x42,0x42,0x42,0x42,0x3C,0x00}}, // U
{0x56, {0x42,0x42,0x42,0x42,0x24,0x24,0x18,0x00}}, // V
{0x57, {0x42,0x42,0x42,0x5A,0x5A,0x66,0x42,0x00}}, // W
{0x58, {0x42,0x24,0x18,0x18,0x18,0x24,0x42,0x00}}, // X
{0x59, {0x42,0x42,0x24,0x18,0x08,0x08,0x08,0x00}}, // Y
{0x5A, {0x7E,0x04,0x08,0x10,0x20,0x40,0x7E,0x00}}, // Z
{0x5B, {0x1C,0x10,0x10,0x10,0x10,0x10,0x1C,0x00}}, // [
{0x5C, {0x40,0x20,0x20,0x10,0x08,0x04,0x02,0x00}}, // backslash
{0x5D, {0x38,0x08,0x08,0x08,0x08,0x08,0x38,0x00}}, // ]
{0x5E, {0x08,0x14,0x22,0x00,0x00,0x00,0x00,0x00}}, // ^
{0x5F, {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7E}}, // _
{0x61, {0x00,0x00,0x3C,0x02,0x3E,0x42,0x3E,0x00}}, // a
{0x62, {0x40,0x40,0x5C,0x62,0x42,0x62,0x5C,0x00}}, // b
{0x63, {0x00,0x00,0x3C,0x42,0x40,0x42,0x3C,0x00}}, // c
{0x64, {0x02,0x02,0x3A,0x46,0x42,0x46,0x3A,0x00}}, // d
{0x65, {0x00,0x00,0x3C,0x42,0x7E,0x40,0x3C,0x00}}, // e
{0x66, {0x0E,0x10,0x10,0x3E,0x10,0x10,0x10,0x00}}, // f
{0x67, {0x00,0x3A,0x46,0x42,0x46,0x3A,0x02,0x3C}}, // g
{0x68, {0x40,0x40,0x5C,0x62,0x42,0x42,0x42,0x00}}, // h
{0x69, {0x08,0x00,0x18,0x08,0x08,0x08,0x1C,0x00}}, // i
{0x6A, {0x04,0x00,0x0C,0x04,0x04,0x04,0x44,0x38}}, // j
{0x6B, {0x40,0x40,0x44,0x48,0x70,0x48,0x44,0x00}}, // k
{0x6C, {0x18,0x08,0x08,0x08,0x08,0x08,0x1C,0x00}}, // l
{0x6D, {0x00,0x00,0x66,0x5A,0x5A,0x42,0x42,0x00}}, // m
{0x6E, {0x00,0x00,0x5C,0x62,0x42,0x42,0x42,0x00}}, // n
{0x6F, {0x00,0x00,0x3C,0x42,0x42,0x42,0x3C,0x00}}, // o
{0x70, {0x00,0x5C,0x62,0x42,0x62,0x5C,0x40,0x40}}, // p
{0x71, {0x00,0x3A,0x46,0x42,0x46,0x3A,0x02,0x02}}, // q
{0x72, {0x00,0x00,0x5C,0x62,0x40,0x40,0x40,0x00}}, // r
{0x73, {0x00,0x00,0x3E,0x40,0x3C,0x02,0x7C,0x00}}, // s
{0x74, {0x10,0x10,0x7C,0x10,0x10,0x12,0x0C,0x00}}, // t
{0x75, {0x00,0x00,0x42,0x42,0x42,0x46,0x3A,0x00}}, // u
{0x76, {0x00,0x00,0x42,0x42,0x42,0x24,0x18,0x00}}, // v
{0x77, {0x00,0x00,0x42,0x42,0x5A,0x5A,0x24,0x00}}, // w
{0x78, {0x00,0x00,0x42,0x24,0x18,0x24,0x42,0x00}}, // x
{0x79, {0x00,0x42,0x42,0x46,0x3A,0x02,0x02,0x3C}}, // y
{0x7A, {0x00,0x00,0x7E,0x04,0x18,0x20,0x7E,0x00}}, // z
{0x7B, {0x0C,0x10,0x10,0x60,0x10,0x10,0x0C,0x00}}, // {
{0x7C, {0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x00}}, // |
{0x7D, {0x30,0x08,0x08,0x06,0x08,0x08,0x30,0x00}}, // }
{0x7E, {0x00,0x00,0x32,0x4C,0x00,0x00,0x00,0x00}}, // ~
};

#endif

```

main.cpp

Programme de test — affichage de texte avec `afficherTexte()`

```

#include "st7920.h"

void setup()
{
    st7920_init();
    effacerEcran();

    // Chaque chaîne est limitée à 16 caractères max (16 × 8 px = 128 px)
    // pour éviter tout débordement et mélange de lignes
    afficherTexte("Hello ESP32!", 0, 0);
    afficherTexte("Ligne 2 texte", 0, 8);
    afficherTexte("0123456789ABCDEF", 0, 16);
}

void loop() {}

```